

LoopLand Park Hackathon

Deque & Circular Linked Lists · Problem Statements

DEVELOPED BY TALENCIAGLOBAL





About This Hackathon

LoopLand is an amusement park where every system loops. Across eight levels you will build its operations: a fast-track ride lane, a carousel caller, a parade palindrome check, a crowd window monitor, a carousel splitter, a sorted maintenance ring, a moving-walkway navigator, and a grand parade dispatcher.

Each level states the inputs, the required outputs and the rules you may rely on. The levels run from intermediate to expert and centre on two structures: the **deque** and the **circular linked list**. No starter code is provided; design and write each solution yourself.

Core Structures

- Deque (double-ended queue)
- Circular Linked List

Difficulty Range

- ★★★ Intermediate
- ★★★★★ Advanced
- ★★★★★★ Expert

Levels Overview

Eight challenges take you from intermediate deque operations to expert capstone problems combining both data structures.

1	Fast-Track Gate ★★★ Intermediate · Deque Priority Lane
2	Carousel Caller ★★★ Intermediate · Circular List Traversal
3	Parade Palindrome ★★★ Intermediate-Advanced · Two-End Comparison
4	Crowd Window Monitor ★★★★ Advanced · Monotonic Deques
5	Split the Carousel ★★★★ Advanced · Slow/Fast Split
6	Maintenance Ring ★★★★ Advanced · Sorted Circular Insert
7	Walkway Navigator ★★★★★ Expert · 0-1 BFS
8	Grand Parade Dispatcher ★★★★★ Expert · Deque + Circular List

Level 1 — Fast-Track Gate

DEQUE · PRIORITY LANE

★★★ INTERMEDIATE

Scenario

LoopLand's headline coaster runs a fast-track lane: VIP guests join at the **FRONT** of the line, regular guests at the **BACK**. The ride boards from the front, and the guest at the very back can give up and walk away. One deque handles all four movements.

Concepts Covered

- A deque as a two-ended waiting line
- Push at the front (VIP) and at the back (regular)
- Pop at the front (boarding) and at the back (quitting)
- Reading the remaining line front-to-back

Formula Guidance

VIP id -> push_front(id)
REGULAR id -> push_back(id)
BOARD -> pop_front (boards the ride)
QUIT -> pop_back (gives up)

Problem Statement

Given: A count n, then n operations: VIP id, REGULAR id, BOARD or QUIT

Required output:

- The guests who boarded, in order
- The guests who quit, in order
- The guests still waiting, front to back

Constraints

- VIP pushes at the front; REGULAR pushes at the back
- BOARD pops the front; QUIT pops the back; both do nothing on an empty lane
- Guest ids are short tokens

Sample Run

Input:

Output:

7

Boarded: 3 1

REGULAR 1

Quit: 2

REGULAR 2

Waiting: 4

VIP 3

BOARD

QUIT

REGULAR 4

BOARD

⚠ BOARD and QUIT act on opposite ends — guard both against an empty lane, and never confuse which end is which.

💡 **Thinking skill:** A priority lane is just a deque seen from the right angle: privilege is an insertion end, not a separate structure.

Level 2 — Carousel Caller

CIRCULAR LINKED LIST · TRAVERSAL

★★★ INTERMEDIATE

Scenario

The carousel's horses stand in a ring, and the operator calls riders to horses one at a time, spinning the ring between calls. Model the carousel as a circular linked list: a `CALL` announces the current horse and advances one place; a `ROTATE` spins past `k` horses silently.

Concepts Covered

- Building a circular singly linked list
- Advancing with `cur = cur.next`, no wrap logic
- Announcing the current node
- Rotations larger than the ring size

Formula Guidance

```
CALL  -> record cur.val ; cur = cur.next
ROTATE k -> repeat cur = cur.next, k times
```

Problem Statement

Given: A horse count `n` and their names, then `m` operations: `CALL` or `ROTATE k`

Required output:

- The horses called, in order
- The horse the carousel rests on afterwards

Constraints

- The last horse links back to the first
- `CALL` outputs the current horse then advances one place; `ROTATE k` advances `k` places with no output
- `k` may exceed `n` — extra laps are harmless

Sample Run

Input:	Output:
4	Called: A D A
A B C D	Current: B
4	
CALL	
ROTATE 2	
CALL	
CALL	

⚠ Close the ring (`last.next = head`) before any traversal — one missing link and a long `ROTATE` walks off the list.

💡 **Thinking skill:** A circular list turns 'spin the wheel' into plain pointer-following: the ring topology absorbs all the modular arithmetic.

Level 3 — Parade Palindrome

DEQUE · TWO-END COMPARISON

★★★ INTERMEDIATE-ADVANCED

Scenario

LoopLand's evening parade should read the same from either park gate. Load the float lineup into a deque and compare it from both ends at once: pop one from the front and one from the back, and count every pair that disagrees.

Concepts Covered

- A deque consumed from both ends
- Pairing the i -th and the $(n-1-i)$ -th elements
- Early-exit versus full-count comparisons
- The middle element of an odd lineup

Problem Statement

Given: One line — the parade lineup (a string of characters)

Required output:

- Whether the lineup is a palindrome
- How many front/back pairs mismatch

Constraints

- Pop the front and the back together while more than one element remains
- A single leftover middle element matches itself
- The lineup is a palindrome exactly when no pair mismatches

Formula Guidance

```
while size > 1:
    a = pop_front
    b = pop_back
    mismatch if a != b

palindrome <=> mismatches == 0
```

Sample Run

Input:
abcxba

Output:
Palindrome: No
Mismatched Pairs: 1

⚠ Pop one element from EACH end per round — taking two from the same end compares neighbours instead of mirrored positions.

💡 **Thinking skill:** Symmetric questions want symmetric access: a deque gives both ends in $O(1)$, so the palindrome check is a single inward sweep.

Level 4 — Crowd Window Monitor

MONOTONIC DEQUES · SLIDING WINDOWS

★★★★ ADVANCED

Scenario

Park control watches crowd counts over every window of k consecutive readings: the peak, the trough, and the largest peak-to-trough spread any window showed. Two monotonic deques — one decreasing, one increasing — answer all of it in linear time.

Concepts Covered

- A monotonic deque of indices
- A decreasing deque for maxima, an increasing one for minima
- Evicting indices that slide out of the window
- Deriving the max spread from paired extremes

Formula Guidance

```
maxima deque: evict back while a[back] <= a[i]
minima deque: evict back while a[back] >= a[i]
evict front when its index <= i - k
spread = max over windows of (max[w] - min[w])
```

Problem Statement

Given: A length n and window size k , then n crowd readings

Required output:

- The maximum of every window
- The minimum of every window
- The largest (max - min) spread across all windows

Constraints

- $1 \leq k \leq n$
- Each index enters and leaves each deque at most once — $O(n)$ total
- Maxima and minima are reported window by window, left to right

Sample Run

```
Input:
8 3
5 1 3 9 2 8 4 6

Output:
Window Maxima: 5 9 9 9 8 8
Window Minima: 1 1 2 2 2 4
Max Spread: 8
```

⚠ The two sweeps need opposite eviction comparisons ($\leq$ for maxima, $\geq$ for minima); reusing the same one silently reports the wrong extreme.

💡 **Thinking skill:** One comparison flip turns a maximum tracker into a minimum tracker — the deque mechanics are identical, only the monotonic direction changes.

Level 5 — Split the Carousel

CIRCULAR LINKED LIST · SLOW/FAST SPLIT

★★★★ ADVANCED

Scenario

A carousel is being split into two smaller rides. Given its ring of seats, divide the circular list into two smaller circular lists – the first taking the extra seat when the count is odd – using the slow/fast pointer technique to find the middle in one pass.

Concepts Covered

- Slow/fast pointers on a circular list
- Locating the middle without counting first
- Re-linking two closed rings
- Verifying circularity after surgery

Formula Guidance

```
slow moves 1, fast moves 2
until fast returns near the head
second half starts at slow.next
close ring 1: slow.next = head
close ring 2: tail2.next = head2
```

Problem Statement

Given: A seat count n , then the n seat labels in ring order

Required output:

- The first half, from its head around its ring
- The second half, from its head around its ring
- Confirmation that both halves are closed rings

Constraints

- $n \geq 2$
- For odd n the first half holds one more seat than the second
- Both result lists must be circular – each tail points back to its own head

Sample Run

```
Input:
5
1 2 3 4 5

Output:
First Half: 1 2 3
Second Half: 4 5
Both Circular: Yes
```

⚠ On an even ring the fast pointer stops one node early – advance it once more before cutting, or the second ring ends up open.

💡 **Thinking skill:** Slow/fast pointers find the midpoint of a ring in one pass with $O(1)$ space – the fast pointer's lap acts as the counter.

Level 6 — Maintenance Ring

SORTED CIRCULAR INSERT

★★★★ ADVANCED

Scenario

LoopLand's overnight maintenance runs on a ring of scheduled checkpoint times that technicians cycle through. New checkpoints must slot into the ring so the times stay sorted around the circle — including the tricky wrap point where the largest time meets the smallest.

Concepts Covered

- A sorted circular linked list
- Insertion between two ordered neighbours
- The wrap point (max meets min)
- Inserting below the minimum or above the maximum

Formula Guidance

```
normal spot:
  cur.val <= v <= cur.next.val

wrap spot:
  cur.val > cur.next.val
  and (v >= cur.val or v <= cur.next.val)
```

Problem Statement

Given: An initial sorted list of n checkpoint times, then m new times to insert

Required output:

- The final ring read from its smallest time
- The ring's size

Constraints

- The ring stays sorted ascending around the circle at all times
- A new minimum or new maximum inserts at the wrap point (between the old max and old min)
- Times are integers; duplicates may appear anywhere among equals

Sample Run

```
Input:
3
10 20 30
3
25 5 35

Output:
Ring: 5 10 20 25 30 35
Size: 6
```

⚠ Handle the seam explicitly: without the wrap-point test, a value smaller than the minimum or larger than the maximum never finds a bracketing pair and the walk loops forever.

💡 **Thinking skill:** A sorted ring has exactly one seam — where max meets min — and every boundary insertion (new smallest, new largest) happens at that single seam.

Level 7 — Walkway Navigator

DEQUE · 0-1 BFS

★★★★★ EXPERT

Scenario

The park's moving walkways each push in one direction: riding with the arrow is free, but stepping any other way costs one unit of energy. Find the minimum energy to get from the park entrance (top-left) to the grand stage (bottom-right) — a 0-1 BFS where the deque replaces a priority queue.

Concepts Covered

- 0-1 BFS on an implicit grid graph
- Zero-cost edges pushed at the deque's FRONT
- Unit-cost edges pushed at the BACK
- Why the deque keeps distances sorted

Formula Guidance

```
w(move) = 0 if move follows cell's arrow
          else 1

relax: if dist[cell] + w < dist[next]:
    update

push next at FRONT when w = 0
push next at BACK  when w = 1
```

Problem Statement

Given: Grid dimensions R and C, then R rows of arrows: `> < ^ v`

Required output: The minimum total energy from the top-left to the bottom-right cell

Constraints

- Moving in the cell's arrow direction costs 0; any other direction costs 1
- Moves are 4-directional within the grid
- Each relaxation pushes front for cost 0 and back for cost 1

Sample Run

```
Input:
4 4
>>>>
<<<<
>>>>
<<<<

Output:
Min Energy: 3
```

⚠ Push zero-cost relaxations at the FRONT, not the back — mixing them with unit-cost moves breaks the sorted-distance invariant and overcounts energy.

💡 **Thinking skill:** With only 0 and 1 edge weights, a deque is a two-bucket priority queue: free moves go to the front, paid moves to the back, and order by distance falls out for free.

Level 8 — Grand Parade Dispatcher

DEQUE + CIRCULAR LIST

★★★★★ EXPERT · CAPSTONE

Scenario

For the grand parade, k boarding stations stand in a ring, each with its own line of guests. The dispatcher walks the ring station by station, boarding one guest per stop — odd-numbered pickups take the **FRONT** of that station's line, even-numbered pickups take the **BACK**. A station found empty closes and is spliced out of the ring. The parade is full when every station has closed.

Concepts Covered

- A circular linked list of stations
- A deque of guests at every station
- A global pickup counter alternating front/back
- Splicing closed stations out of the ring

Formula Guidance

```
odd pickup -> station.line.pop_front()
even pickup -> station.line.pop_back()
empty on arrival ->
    before.next = cur.next (station closes)
```

Problem Statement

Given: A station count k , then one line per station: its name, its guest count, and its guests front-to-back

Required output:

- The full boarding order
- The order in which stations closed

Constraints

- Pickups are numbered globally; odd pickups take the line's front, even pickups its back
- A station is closed (and spliced out) when the dispatcher arrives to find it empty; closing does not advance the pickup counter
- After every pickup or closure the dispatcher moves to the next station in the ring

Sample Run

```
Input:
3
A 3 1 2 3
B 1 4
C 2 5 6

Output:
Boarding Order: 1 4 5 3 6 2
Stations Closed: B C A
```

⚠ Only an actual pickup advances the odd/even counter — counting a closure as a step shifts every later front/back choice; and splice with the trailing pointer or the ring breaks.

💡 **Thinking skill:** The capstone runs both structures at once: the ring decides WHERE the dispatcher stands, the deques decide WHO leaves each line — and the alternating rule exercises both deque ends under a moving ring.